

Homework 17: A Scheduler That Survives March

Build something that handles dates correctly, then reproduce four `Math` traps.

Every line your program prints must be identical on every run, and identical regardless of which time zone the machine is set to. That means a fixed `Clock`, fixed dates, and a seeded generator. Output that varies fails this assignment.

Part One: Monthly Billing

Create `Scheduler.java` in the classic form.

A customer signs up on **31 January 2026**. Read that date from a fixed `Clock`, not from `LocalDate.now()` with no argument.

Print their next four billing dates **two ways**:

1. **Naively**, by repeatedly adding one month to the previous result
2. **Anchored**, by always computing from the original signup date

The two disagree from the second month onward.

Explain It

Write a comment covering:

- What `plusMonths` does to 31 January, and why that part is correct
- Why the naive version's error is **permanent** rather than a one-off
- What the anchored version does differently, in one sentence

Part Two: A Meeting Across Daylight Saving

Schedule a 09:00 New York meeting on **Friday 6 March 2026**, then move it three days forward two ways:

- `plusDays(3)`
- `plusHours(72)`

Print both, plus the real hours elapsed between the original and the `plusDays` result. It will not be 72.

Then show what London sees for both moments.

Three More Cases

1. **The ambiguous hour.** 01:30 on 1 November 2026 in New York happens twice. Print both, and how far apart they are.
2. **The nonexistent hour.** 02:30 on 8 March 2026 never happens. Print what `atZone` does with it.
3. **The shifting gap.** London and New York change on different dates. Say what that means for anyone who hard-codes a five-hour difference.

Explain It

A comment covering why `plusDays` and `plusHours` differ, which question each one answers, and which you would use for a recurring meeting versus a timeout.

Part Three: Four Math Traps

Reproduce and explain each:

1. `Math.abs(Integer.MIN_VALUE)` and the **negative bucket index** it causes. Show `abs(hash) % buckets` alongside `floorMod(hash, buckets)`.
2. `-7 / 2` versus `Math.floorDiv(-7, 2)`
3. `Math.round(2.5)` versus `Math.round(-2.5)`
4. Silent overflow, and what `Math.addExact` does instead

Careful with trap 1. Pick your bucket count so the bug actually shows. Some values make both forms agree and hide it entirely, and working out which is part of the exercise.

Part Four: Reproducible Sampling and Formatting

Write a method that picks N names at random from a list, taking a **seed** as a parameter.

Call it twice with the same seed and once with a different one. Show the first two agree.

Write a comment on what an unseeded generator would do to a test, and name one thing you must never use these generators for.

Finish with a small formatted report using `formatted`, with:

- A date formatted through an explicit `Locale`
- A left-aligned label column
- A thousands-separated number

Say in a comment why the `Locale` is not optional.

Acceptance Criteria

- ☐ Runs with `java Scheduler.java`
- ☐ Running it twice produces byte-identical output
- ☐ A fixed `Clock` is used; no bare `now()` anywhere
- ☐ Both billing schedules are printed and disagree from month two
- ☐ A comment explains why the naive error is permanent
- ☐ `plusDays(3)` and `plusHours(72)` are shown giving different results
- ☐ Real elapsed hours are printed and are not 72
- ☐ The ambiguous hour is shown twice, an hour apart
- ☐ The nonexistent hour is shown being moved forward
- ☐ All four Math traps are reproduced with visible output
- ☐ Trap 1's bucket count actually exposes the negative index
- ☐ Sampling is seeded, and two same-seed runs agree
- ☐ `ofPattern` is given an explicit `Locale`, with a comment on why

Stretch

Write `List<ZonedDateTime> weeklyMeetings(ZonedDateTime first, int count)` returning a weekly recurrence that stays at the same **local** time across a daylight saving transition.

Then write a comment on what would go wrong if you stored each occurrence as an `Instant` computed by adding seven days' worth of seconds, and what you would store instead for a meeting a year from now whose zone rules might change before it happens.

Hint, if your output changes between runs

Search for `now()` with no argument, and for any generator created without a seed. Those are the only two sources of variation here, and both have a form that takes the thing they depend on as a parameter. That is not a trick for homework; it is how time-dependent code is made testable in general.